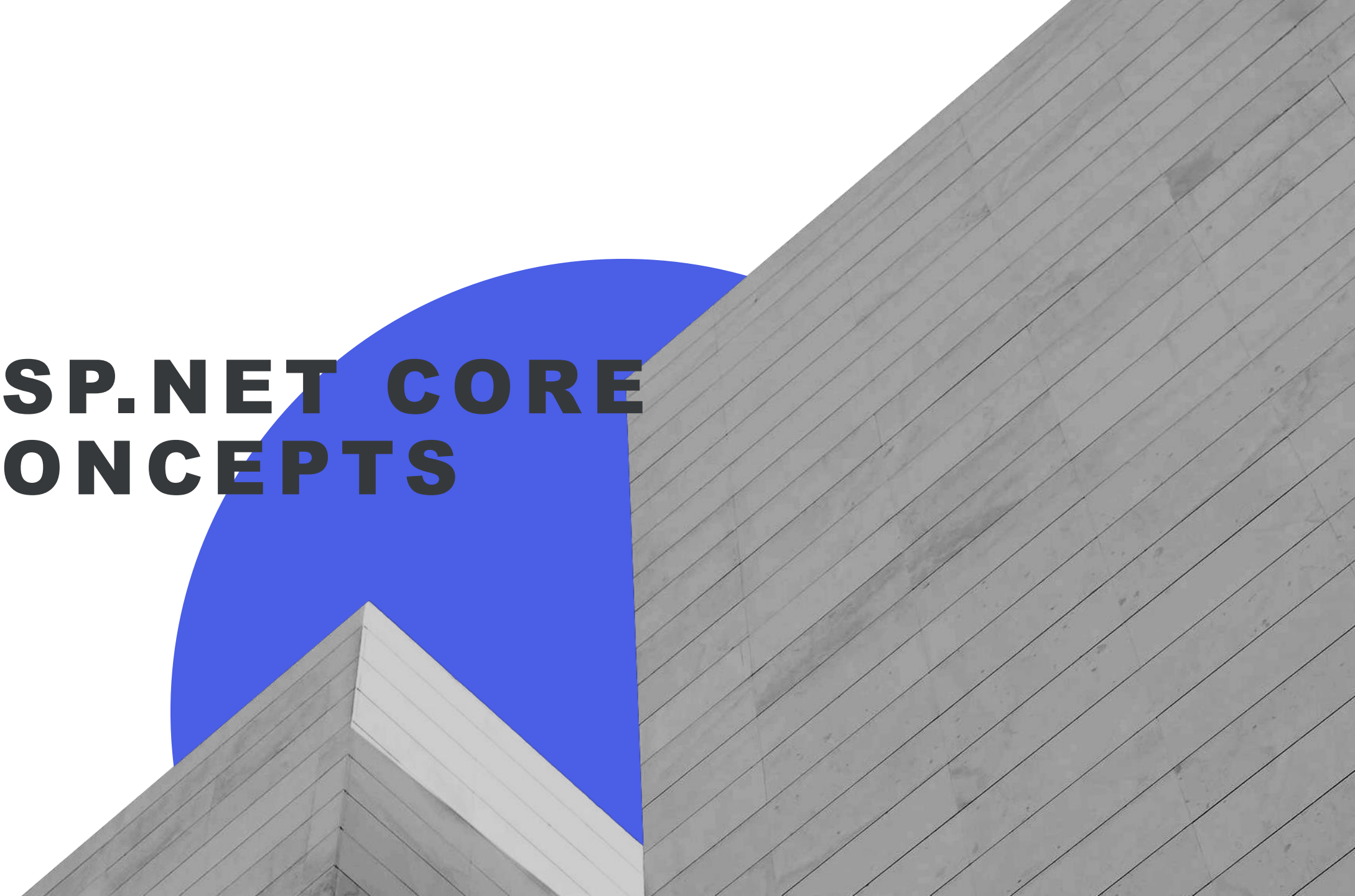
ASP.NET CORE CONCEPTS



1. Dependency Injection (DI) Lifecycle

What is it?

Dependency Injection is a design pattern used to achieve **loose coupling** between classes by providing dependencies externally instead of creating them inside the class.

- Why is DI needed?

Without DI:

- Classes become **tightly coupled**
- Hard to **test**
- Difficult to **maintain and extend**

With DI:

- Code becomes **loosely coupled**
- Easy to **replace components**
- Supports **unit testing (mocking)**

ADVANTAGES OF DEPENDENCY INJECTION (SINGLETON, SCOPED, TRANSIENT)

- **LOOSE COUPLING** → CLASSES ARE INDEPENDENT AND EASIER TO MODIFY
- **BETTER TESTABILITY** → EASY TO USE MOCK OBJECTS FOR UNIT TESTING
- **REUSABILITY** → SERVICES CAN BE REUSED ACROSS THE APPLICATION
- **MAINTAINABILITY** → CHANGES IN ONE CLASS DON'T AFFECT OTHERS MUCH
- **FLEXIBILITY** → EASY TO SWITCH IMPLEMENTATIONS (E.G., DB, SERVICES)

DISADVANTAGES

- **DEBUGGING DIFFICULTY** → HARDER TO TRACE OBJECT CREATION FLOW
- **MEMORY ISSUES** → SINGLETON CAN HOLD MEMORY FOR ENTIRE APP LIFETIME
- **COMPLEXITY** → HARD TO UNDERSTAND FOR BEGINNERS
- **OVERHEAD** → SLIGHT PERFORMANCE COST DUE TO OBJECT RESOLUTION
- **OVERUSE PROBLEM** → TOO MANY ABSTRACTIONS CAN MAKE CODE CONFUSING

Type	Lifetime	Use Case
Singleton	Entire app	Logging, config
Scoped	Per request	Database operations
Transient	Every use	Lightweight services

What is IConfiguration?

- **IConfiguration** is an interface in ASP.NET Core used to **read configuration settings** from different sources.
- 👉 It acts as a **central access point** for all application settings.
- Why is IConfiguration needed?

Without IConfiguration:

- You would hardcode values (like DB connection, API keys)
- Changing settings requires recompiling code

With IConfiguration:

- Settings are externalized (kept outside code)
- You can change values without modifying code

Purpose

- To **separate configuration from code**
- To support **multiple environments**
- To make applications **flexible and scalable**

What is appsettings.json?

- **appsettings.json** is the default configuration file in ASP.NET Core used to store application settings in JSON format.
- It is mainly used to keep:
  - Connection strings
  - API keys
  - Logging settings
  - Custom application values
- It helps in separating configuration from code, so changes can be made without modifying or recompiling the application.
- Supports **environment-based configuration** using files like:
  - appsettings.Development.json
  - appsettings.Production.json

Key Benefits

- Easy to read and modify
- Centralized configuration
- Supports multiple environments
- Improves maintainability

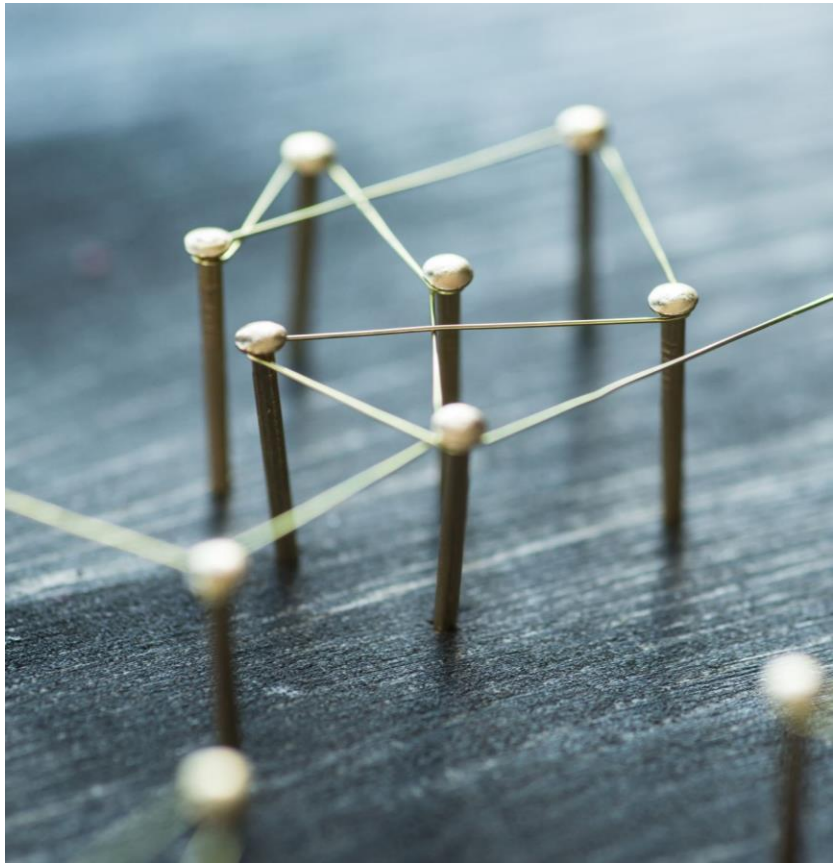
- **Logging in ASP.NET Core**

- **Logging** is the process of recording application events, errors, and runtime details.
- It helps in:
  - Debugging issues
  - Monitoring application performance
  - Tracking errors in production
- ASP.NET Core provides a **built-in logging framework** with support for **Dependency Injection**.
- Supports **log levels**:
  - Trace, Debug, Information, Warning, Error, Critical
  - Supports multiple **logging providers**:

  Console, Debug, EventSource, File (via external libraries)

- Logging Frameworks / Tools

- **Serilog** → Structured logging (JSON, easy analysis)
- **NLog** → Flexible and widely used logging library
- **Log4Net** → Older but stable logging framework



- **Session State**
  - **Session State** is used to store **user-specific data across multiple requests** in a web application.
  - Since HTTP is **stateless**, session helps remember things like:
    - Logged-in user
    - Shopping cart
    - User preferences
  - 👉 Each user is identified using a **unique Session ID** (usually stored in cookies).

  - Why it is important
  - Without session:
  - Application cannot remember user data between requests
- With session:
  - Enables **continuous user experience** (login stays, cart stays)

GLOBALIZATION & LOCALIZATION

- GLOBALIZATION:

Globalization is the process of designing an application so that it can support users from different countries, languages, and cultures. Instead of building separate applications for each region, we build one application that is flexible enough to work everywhere.

- LOCALIZATION:

Localization comes after that. It is the process of customizing the application for a specific region. This includes translating text into a local language, changing date and time formats, currency symbols, and number formats based on that region.

- **“Localization depends on globalization.”**

- **What is API Versioning?**

- API Versioning is the practice of **managing changes in an API** by assigning different versions, so that existing clients continue to work even after updates.

- Why API Versioning is Important

- Prevents breaking existing applications
- Allows adding new features safely
- Maintains backward compatibility
- Supports multiple clients using different versions

Common Methods of API Versioning

1. URL Versioning

- Version is included in the URL
👉 Example:
/api/v1/products

2. Query String Versioning

- Version is passed as a parameter
👉 Example:
/api/products?version=1

3. Header Versioning

- Version is sent in request headers
👉 Example:
api-version: 1

4. Media Type Versioning

- Version is specified in content type
👉 Example:
Accept: application/json;v=1

- **Swagger in ASP.NET Core**

- Swagger in ASP.NET Core is used to automatically generate interactive API documentation for Web APIs. It allows developers to view, test, and understand API endpoints directly from a browser.

How Swagger Works

- Scans API controllers and endpoints
- Generates API documentation (OpenAPI format)
- Displays it using **Swagger UI**
- Allows testing APIs without external tools

Key Components

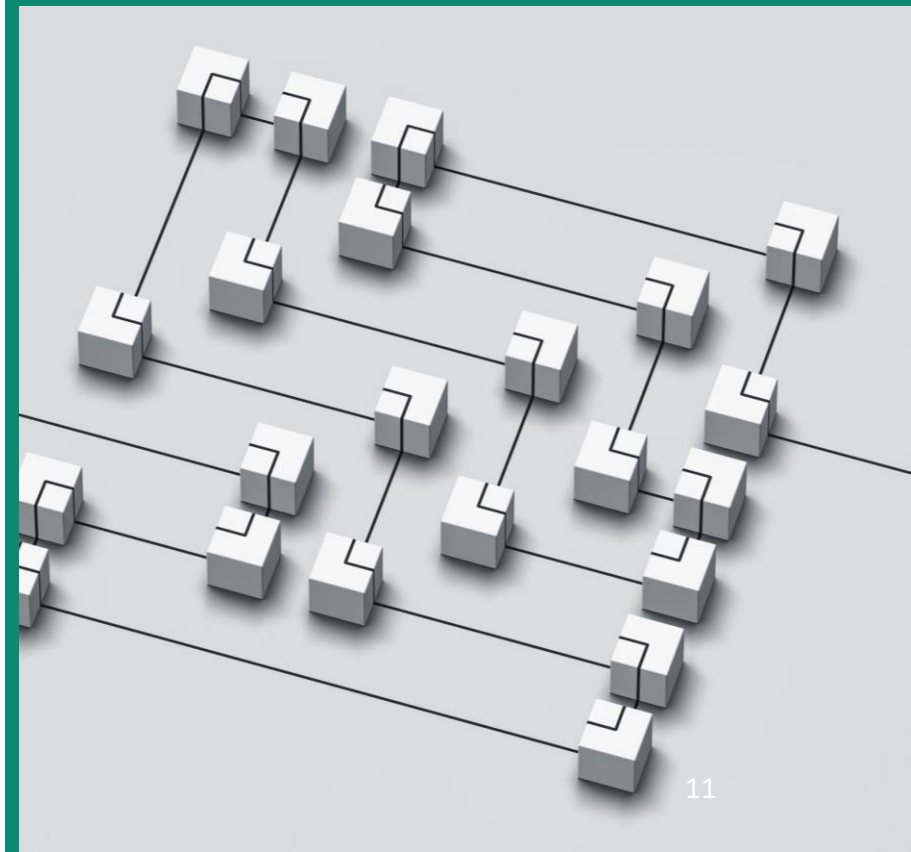
- **Swagger (OpenAPI Specification)** → Describes API structure
- **Swagger UI** → Web interface to test APIs
- **Swashbuckle** → Library used to integrate Swagger in ASP.NET Core

Minimal APIs in ASP.NET Core?

- Minimal APIs are a feature in ASP.NET Core that allow developers to build HTTP APIs with minimal code and without using Controllers. They provide a lightweight and streamlined approach to creating web APIs.

Key Characteristics

- Uses a **single file (Program.cs)** for configuration and endpoints
- Eliminates the need for **Controllers and Startup classes**
- Supports **routing, dependency injection, and middleware**
- Designed for **simplicity and performance**
- How It Works
- In Minimal APIs, endpoints are defined directly using methods like:
- MapGet()
- MapPost()
- MapPut()
- MapDelete()
- These methods map HTTP requests directly to code.



What is Web API?

- Web API is a framework used to build **HTTP services** that expose data to different clients such as web applications, mobile apps, and other services. It mainly returns data in formats like **JSON or XML**.

What is MVC?

- MVC (Model-View-Controller) is a design pattern used to build web applications with user interfaces. It handles both data processing and UI rendering (HTML views).

When to Use

Use Web API when:

- You are building backend services
- Data needs to be consumed by multiple clients
- Developing microservices architecture

Use MVC when:

- You are building a web application with UI
- You need server-side rendering
- Application is mainly for browser users

Feature	Web API	MVC
Purpose	Build RESTful services	Build web applications
Output	JSON / XML (data only)	HTML (Views)
User Interface	Not supported	Supported (Razor Views)
Client Type	Mobile apps, SPA, external systems	Web browsers
Base Class	ControllerBase	Controller
View Support	No	Yes
Routing	Attribute routing	Conventional + Attribute routing

What is Serilog?

- **Serilog** is a popular third-party logging library used in ASP.NET Core for **structured logging**.
- Unlike traditional logging, it stores logs in a **structured format (like JSON)** instead of plain text.
- It integrates easily with ASP.NET Core's built-in logging system.

Key Features

- **Structured logging** → logs stored as key-value pairs
- Supports multiple **outputs (sinks)**:
  - File
  - Database
  - Console
  - Cloud tools (like Elasticsearch)
- Easy filtering and searching of logs

Purpose

- To make logs **easy to analyze and query**
- To improve **monitoring and debugging**
- To support **production-level logging systems**

.NET FRAMEWORK VS .NET CORE

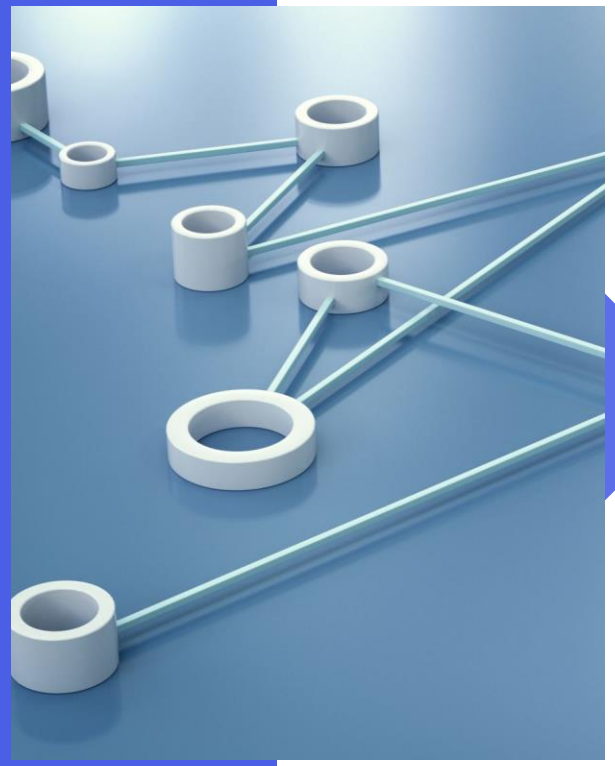
Feature	.NET Framework	.NET Core
Platform	Windows only	Cross-platform (Windows, Linux, macOS)
Open Source	❌ No	✅ Yes
Performance	Moderate	High performance
Deployment	System-wide	Self-contained
Scalability	Limited	Highly scalable
Use Case	Desktop apps, legacy apps	Web apps, APIs, cloud, microservices
Installation	Pre-installed on Windows	Needs to be installed

What is .NET Framework?

- .NET Framework is a Windows-only development platform used to build desktop and web applications. It is older and mainly used for traditional enterprise applications.

What is .NET Core?

- .NET Core is a modern, open-source, cross-platform framework used to build web, cloud, and microservices-based applications. It runs on Windows, Linux, and macOS.



- **Advantages**
- ✅ **.NET Framework**
  - Strong support for Windows desktop apps (WinForms, WPF)
  - Mature and stable for legacy systems
- ✅ **.NET Core**
  - Cross-platform support
  - Better performance and scalability
  - Lightweight and flexible
  - Ideal for **modern development (cloud & APIs)**

Caching in ASP.NET Core?

- **Caching** is a technique used to store frequently accessed data temporarily to **improve performance**.
- Instead of fetching data repeatedly from the database, cached data is returned quickly.
- It reduces **server load** and improves **response time**.

Types of Caching

- **In-Memory Caching** → Stored in application memory (fast, single server)
- **Distributed Caching** → Stored in external systems like Redis (shared across servers)

Purpose

- To **increase application speed**
- To **reduce database calls**
- To improve **scalability and performance**

Advantages

- Faster response time
- Reduces database and API load
- Improves user experience
- Enhances scalability

Disadvantages

- Data can become outdated (stale data issue)
- Cache invalidation is complex
- Requires additional memory/resources
- Distributed cache needs extra setup

in-Memory Caching?

- **In-Memory Caching** stores data directly in the **server's memory (RAM)** for fast access.
- Data is retrieved from memory instead of querying the database repeatedly.
- It is mainly used in **single-server applications**.

Purpose

- To **improve performance and speed**
- To reduce **database load**
- To provide **quick data access**

Key Points

- Very **fast** (no network calls)
- Data is **temporary** (lost if app restarts)
- Not shared across multiple servers

What is Distributed Caching?

- **Distributed Caching** stores cached data in an external system (like Redis, SQL Server) instead of application memory.
- It allows multiple application instances to share the same cached data.
- Commonly used in scalable and cloud-based applications.

Purpose

- To support multi-server applications
- To maintain consistent data across instances
- To improve scalability and performance
- 🔧 Key Points
- Data is stored outside the application
- Shared across multiple servers
- Slightly slower than in-memory (due to network call)

Advantages

- Scalable and suitable for distributed systems
- Data consistency across servers
- No data loss on app restart
- Supports large-scale applications

Disadvantages

- Requires setup (Redis, SQL Server, etc.)
- Network latency may reduce speed
- More complex than in-memory caching
- Additional infrastructure cost

THANK YOU

PRESENTED BY
LAVANYA